Technical Design Document (TDD)

Project: Researcher Knowledge Base and Ranking System

Version: 1.0 (Current-state, till now)

Date: 2026-05-07

1. Technical Scope

This document describes the implemented technical architecture, module responsibilities, data flow, runtime behavior, and deployment considerations for:

- Offline ETL pipeline in `researcher-kb-pipeline`
- Online FastAPI ranking service

2. High-Level Architecture

2.1 Offline Pipeline (Batch)

- Stage 1: Discover papers from OpenAlex `/works`
- Stage 2: Extract authors and aggregate per-concept stats
- Stage 3a: Enrich author profiles (OpenAlex + Semantic Scholar)
- Stage 3b: Enrich coauthor/mentorship signals
- Stage 3c: Enrich institution profiles
- Stage 4: Assemble final researcher profile + MySQL upserts + optional exports

2.2 Online Ranking API (Request/Response)

- `rank_service.py` exposes FastAPI endpoints.
- `search.py` orchestrates retrieval + scoring.
- `ranking.py` computes Q/H/final ranking and explainability payloads.

3. Code Modules and Responsibilities

- `pipeline.py`: CLI entrypoint and orchestration, including incremental workflow support and staged execution.
- `stage1_discover.py`: OpenAlex work discovery with filters and pagination.
- `stage2_extract.py`: author-level extraction and concept aggregation.

- `stage3_enrich.py`: profile/coauthor/institution enrichment stages.
- `stage4_assemble.py`: profile assembly, sanitization, batched DB upserts, artifact exports.
- `db.py`: DB connection + SQL upsert/fetch operations.
- `rank_service.py`: API schema models, endpoint handlers, error mapping.
- `search.py`: ranking pipeline composition for live and mock modes.
- `ranking.py`: scoring algorithms, Pareto option, reason/contribution builders.
- `config.py`: runtime constants and scoring defaults.

4. Data Flow (Pipeline)

4.1 Stage Outputs

- Stage 1 output: raw papers JSONL per concept in `data/raw/papers`.
- Stage 2 output: `data/intermediate/field_author_map.json`.
- Stage 3 outputs: profile/coauthor/institution JSON files in `data/raw/*`.
- Stage 4 output:
  - `data/knowledge_base.json`
  - `data/knowledge_base_summary.xlsx`
  - `data/knowledge_base_summary_schema.json`

4.2 Stage 4 Persistence Flow

For each researcher in a batch:

1. Build unified profile object.
2. Sanitize institution/researcher records using required/default/null rules.
3. Upsert institution and researcher rows.
4. Upsert related papers/topics/collaborations.
5. Use per-researcher SQL savepoint for partial failure containment.
6. Commit batch transaction; rollback on batch-level failures.

5. API Design

5.1 Endpoints

- `GET /health`
  - Response: `{ status, service }`
- `POST /rank`
  - Request fields include:
    - required: `query`

    - filters: `region`, `institution_id`
    - controls: `limit`, `pareto_enabled`
    - ranking knobs: `top_k`, `min_unique_researchers`, `max_top_k`, `target_papers_per_researcher`, `decay_lambda`, `max_papers_per_researcher`, `recency_lambda`, `citation_beta`
    - execution mode: `use_mock_data`, `mock_data_file`, `use_simple_ranking`

5.2 Response Shape

- `results[]` with `researcher_id`, `H`, `Q`, `final_score`, `reason`, `contribution`, `components`
- `pareto` metadata (`enabled`, dominated IDs and map)
- `debug` diagnostics from retrieval/scoring pipeline

5.3 Error Mapping

- `400`: value/validation style failures
- `503`: dependency/config readiness issues (DB env vars, Pinecone key, embedding failures)
- `500`: unclassified server errors

6. Scoring Design

6.1 Final Score

Current formula:

- Final = wH _ H + wQ _ Q
- Weights come from configuration (`H_WEIGHT`, `Q_WEIGHT`).

6.2 H Score Modes

- Simple mode (default):
    - `H = min(h_index / 100, 1.0)`
- Advanced mode:
    - Normalized weighted combination of:
        - `h_index`
        - `log1p(total_citations)`
        - `quality_score`
        - `recency_score`
        - `seniority_score`

6.3 Q Score Modes

- Simple mode (default):
  - Average of top-N paper similarity scores per researcher, zero-padded when fewer than N papers.
- Advanced mode:
  - Similarity adjusted by rank decay, recency decay, citation boost.
  - Per-researcher weighted average of adjusted paper scores.

6.4 Candidate Retrieval and Enrichment

- Pinecone query starts at `top_k` and may expand (doubling) until diversity/depth goals or limits are reached.
- Optional depth enrichment performs researcher-filtered Pinecone queries to improve per-researcher paper coverage.

6.5 Optional Pareto Pruning

- Epsilon-Pareto filtering can remove dominated candidates using H/Q metric comparison.

7. Runtime and Configuration

7.1 Key Runtime Dependencies

- OpenAlex API
- Semantic Scholar API
- MySQL
- Pinecone
- Embedding provider used by `embeddings.py`

7.2 Important Config Areas (`config.py`)

- Concept/topic IDs and labels (`CONCEPTS`)
- Rate limiting (`OPENALEX_SLEEP`, `S2_SLEEP`)
- Staleness refresh window (`STALENESS_DAYS`)
- Ranking defaults (`USE_SIMPLE_RANKING`, weights, top-k/depth controls)

8. Data Model (Logical)

8.1 Core Entities

- Researcher
- Institution
- Paper

- Topic
- Collaboration

8.2 Researcher Profile Sections (Assembled JSON)

- Identity/external IDs
- Global metrics
- Affiliation current/history
- Broad and granular topics
- Citation trend
- Field relevance per concept
- Lab/mentorship signals
- Institution quality
- Papers grouped by concept
- Semantic Scholar enrichment

9. Transaction and Fault Tolerance

- Stage 4 uses SQL savepoints per researcher record to prevent full-batch failure on isolated data issues.
- Batch-level exceptions trigger rollback and propagate error.
- API layer captures runtime exceptions and returns mapped HTTP status.

10. Observability and Logging

- Pipeline logs progress and counters by stage and batch.
- API logs request and result counts.
- Debug block in ranking response includes retrieval/scoring internals (candidate counts, top-k expansion, enrichment metadata).

11. Deployment Notes

- Local: run pipeline with Python CLI; run API via Uvicorn.
- Containerized deployment supported via Dockerfiles and cloud build config.
- Cloud Run and Cloud SQL patterns are reflected in repository deployment assets.

12. Known Risks and Technical Debt

- Source-data inconsistency can cause profile sparsity or metadata mismatches.
- Ranking quality depends on embedding quality and Pinecone coverage.
- Large concept lists can increase ingestion latency and API-source pressure.
- Incremental consistency between DB and vector index requires ongoing validation.

13. Verification Checklist (Current)

1. Pipeline stage execution works individually and end-to-end.
2. Stage 4 commits valid records while skipping malformed records safely.
3. API health endpoint responds consistently.
4. Ranking endpoint returns deterministic structure for both mock and live modes.
5. Debug metadata exposes enough signals to troubleshoot retrieval depth and filtering.

14. Next Technical Improvements

- Add automated integration tests for stage boundaries and rank response contracts.
- Add schema and contract validation for artifact outputs.
- Add periodic DB/Pinecone consistency audit job.
- Add stricter config validation at startup.
- Introduce centralized metrics (latency, success rate, throughput, error cardinality).